SYS-304 Milestone 4: Continual Learning & Final Demo

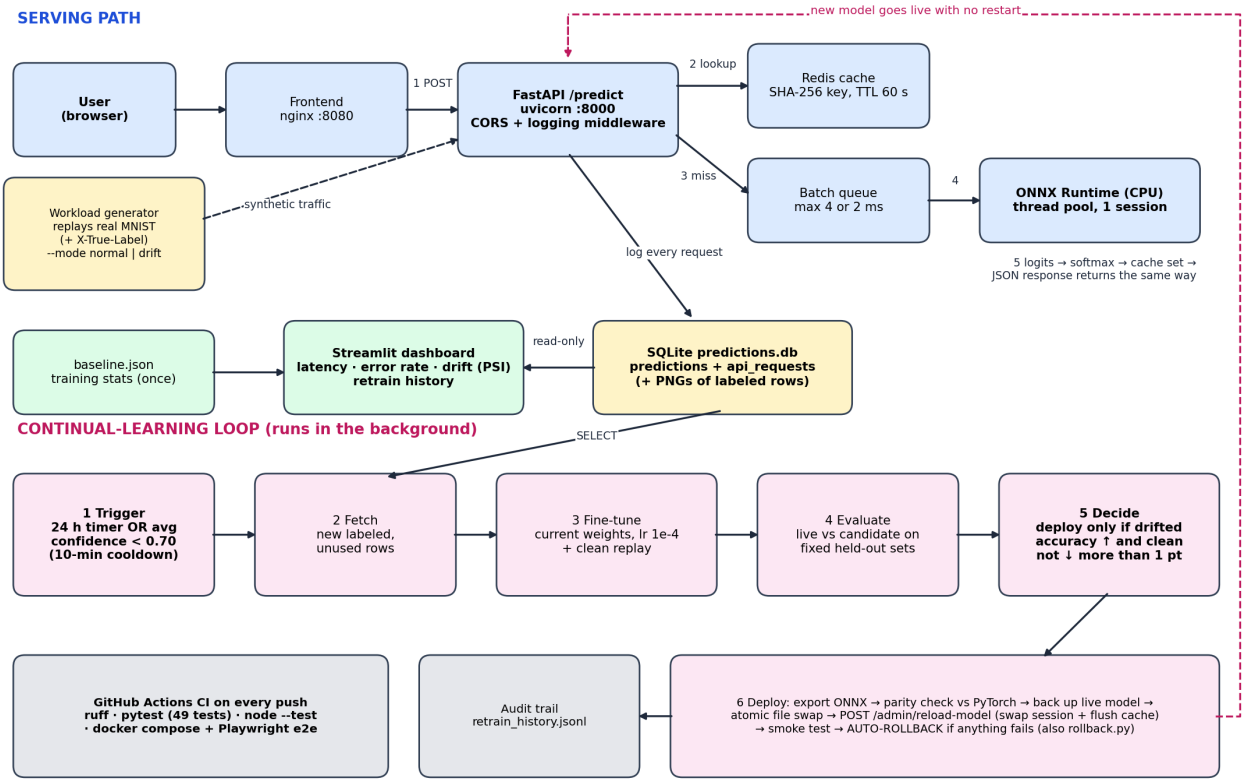
Design Document: Digit Recognizer service (MNIST, ResNet18 to ONNX)

1. Domain and problem

The service classifies a hand-drawn digit (0-9) from an uploaded image. Milestones 1-3 built and optimised the *serving* side: a ResNet18 trained on MNIST, exported to ONNX (5.4x faster than naive PyTorch at identical 97.52% accuracy), behind FastAPI with exact-match Redis caching and dynamic request batching.

Milestone 4 addresses what happens *after* deployment. A model is trained on a fixed snapshot, but real users draw differently (thicker strokes, tilted, noisy cameras, different contrast). Accuracy silently decays while the service still returns confident-looking answers. The problem is therefore a closed loop: **observe** what the service receives and returns, **detect** when inputs or behaviour move away from training, **retrain** on new labelled data, and **redeploy safely**, without taking the service down and without letting a bad model reach users.

2. System architecture



Data flow, user to model and back. (1) The browser uploads the image through the nginx frontend to FastAPI. (2) The SHA-256 of the image bytes is looked up in Redis; a hit returns immediately. (3) On a miss the image is preprocessed (grayscale, 28x28, invert light backgrounds, scale to 0-1) and queued. (4) The batch worker groups up to 4 requests or waits 2 ms, then runs one ONNX inference on a thread pool. (5) Softmax gives digit, confidence and probabilities; the result is cached (60 s), returned, and **logged** to SQLite together with latency, pixel statistics and (for labelled traffic) the true label and image. A middleware also logs every /predict call's HTTP status, so failures that never reach the model are still counted.

3. Design decisions

3.1 Week 7: observability

Decision	Choice	Why / alternative rejected
Request log store	SQLite file with two tables: **predictions** (model view) and **api_requests** (HTTP view).	Zero-ops, shared as a file between container and host. Postgres/MySQL would be the production choice (see bottlenecks). Two tables because a 400 never reaches the model: one table cannot give a true error rate.
Dashboard	Streamlit reading the DB read-only, four sections: API health, model behaviour, drift, retrain history.	Dashboard is a separate process that never calls the API, so monitoring cannot take down serving. Grafana needs a metrics pipeline (Prometheus) that is overkill at this scale.
Latency / errors	p50/p95/p99 and error rate from api_requests; 4xx and 5xx counted separately.	Percentiles, not means: tail latency is what users feel. 5xx = we failed; 4xx = caller sent bad input.
Drift statistic	**PSI** (decile bins taken from training quantiles, flag at >0.25) on mean and std of preprocessed pixels, plus a sample-size-aware z-test. Needs >= 30 samples.	Baseline is computed once, offline, from 2000 real MNIST training images through the *same* preprocess() the service uses. A z-score using per-image spread would barely register real drift, so the z-test divides by std/sqrt(n). PSI compares whole distributions, not only the mean.
Cache hits and drift	Hits skip preprocess(), so they log NULL pixel stats; drift uses cache misses only.	Keeps the cache fast. Acceptable because exact repeats are rare for hand-drawn digits.
Workload generator	Replays real MNIST test images (held-out 0-999 reserved), optional drift (rotation, noise, brightness), configurable rate/labels/bad-request rate.	Needed twice: to produce load/drift for monitoring, and as the only source of *labelled* traffic (real users never send a label).

3.2 Week 8: retraining and deployment

Decision	Choice	Why
Trigger	Elapsed time (24 h) OR average confidence over the last hour < 0.70, with a 10-minute cooldown; `--force` simulates a trigger for demos.	Matches the brief (time threshold or simulated confidence drop). Cooldown stops a persistently low-confidence period from retraining in a loop.
Ground truth	Labels come only from workload-generator traffic (`X-True-Label`); image bytes stored only for those rows.	The brief allows assuming labels arrive later. Storing images for every request would waste space on rows that can never be used.
Fine-tuning	Continue from the *currently deployed* weights, lr 1e-4, 2 epochs, plus replay of clean MNIST training images (1:1).	Small steps and replay limit catastrophic forgetting. Training from scratch would discard prior knowledge and cost far more.
Evaluation	Fixed held-out MNIST test[0:1000], measured clean and through a seeded drift transform. Live model and candidate scored on the same sets.	Comparable across cycles; generator never samples those images, so no train/eval leakage.
Deploy rule	Deploy iff drifted accuracy improves by >= 0.2 pt AND clean accuracy drops by no more than 1 pt.	Rewards adapting to new data but refuses a model that forgot the old distribution.
Deployment mechanics	Export to a separate file, ONNX-vs-PyTorch parity check, back up live model+weights, atomic os.replace, POST /admin/reload-model, smoke test, **automatic rollback** on failure. Audit line appended to retrain_history.jsonl.	Swapping a file does nothing to a running process (session is loaded at startup), so the endpoint reassigns the session. It also flushes Redis: otherwise repeated images keep returning the old model's cached answers.

4. Benchmark recap (Milestone 3, carried forward)

Variant	Latency	Speedup	Accuracy
Naive PyTorch	104.5 ms	1.0x	97.52%
ONNX FP32 (served)	19.3 ms	5.4x	97.52%
ONNX INT8 dynamic	49.1 ms	2.1x	not measured
Distilled student	1.6 ms	67.3x	97.06%

Concurrent load (20 requests)	Total wall time	Throughput
Baseline (blocking)	689.0 ms	29.0 req/s
Thread-pool only	1042.6 ms (worse)	19.2 req/s
Dynamic batching (4 / 2 ms)	577.2 ms	34.6 req/s

The served model is ONNX FP32: the student is faster but the full model was kept for accuracy. Thread-pool offload alone made things slower because per-request dispatch overhead dominated; batching amortises it (20 requests became 6 batches).

5. Scaling bottlenecks and mitigations

Bottleneck	What happens	Candidate mitigation
Single-process CPU inference	One ONNX session on one machine caps throughput (~35 req/s measured under batching). At 100x traffic the queue grows and latency explodes.	Horizontal scaling: N stateless replicas behind a load balancer, autoscaled on queue depth/CPU; multiple uvicorn workers per host; GPU or Triton for larger models; rate limiting and bounded queues (reject with 429/503 rather than melt down); the 67x distilled model as a cheap fallback tier.
Batching window vs tail latency	Waiting 2 ms to fill a batch helps throughput but adds latency at low load.	Adaptive window (shrink when the queue is empty); tune BATCH_MAX_SIZE with p99 from the dashboard.
SQLite logging	Single writer, a new connection and a synchronous write per request on the hot path; one lock for all replicas; image BLOBs bloat the file.	PostgreSQL; batched or asynchronous inserts through a queue (Kafka/Redis stream); sample low-value rows; keep images in object storage.
Redis single instance	Single point of failure; flushing on deploy empties the cache for every user at once.	Redis replicas/cluster; version the cache key by model version so a new model simply misses old entries instead of a global flush.
Hot reload hits one process	/admin/reload-model swaps the session in only the process that received the call. With several workers or replicas the rest keep serving the old model.	Model registry plus version polling by each replica, or rolling restart; canary the new version on a slice of traffic first.
Retraining competes with serving	Training on the serving host steals CPU and raises latency.	Run retraining on a separate node/job (scheduler, off-peak), publish artefacts to object storage.
Label scarcity	Real users send no labels, so retraining relies on generator traffic here.	Human-in-the-loop labelling of low-confidence samples, user feedback buttons, delayed ground truth joins, pseudo-labelling.
Coarse drift signal	Pixel mean/std only detects gross shifts.	KS tests, per-pixel or embedding-space drift, drift on prediction distribution and confidence.
Dashboard on the DB	Dashboard queries grow with table size.	Metrics to Prometheus + Grafana with alerting; pre-aggregated rollups.

6. Failure scenarios and recovery

Scenario	Protection and recovery
A bad model is automatically deployed	Layers: (1) candidate must beat live on drifted data and not regress clean accuracy by >1 pt; (2) ONNX output must match PyTorch before swap; (3) post-deploy smoke test through the real serving path (cache just flushed, so it cannot be a stale hit); (4) any failure triggers automatic restore of the backed-up model; (5) `python retrain/rollback.py` restores the previous model manually; (6) every decision is in the audit trail. Remaining gap: a model that passes held-out tests but is worse on real traffic. Mitigation: canary/shadow deployment and monitoring confidence after deploy.
Traffic spikes 100x overnight	Queue and thread pool saturate, p95/p99 rise and 5xx appear in the dashboard. Mitigation: autoscale replicas, shed load with rate limits/429, rely on cache for repeats, fall back to the distilled model, move logging off the hot path.
Redis down	Cache lookups fail; the service should degrade to compute-only. Mitigation: catch Redis errors and bypass the cache; replicate Redis.
Retrain server unreachable at deploy	Files are updated and the outcome is recorded as deployed_server_unreachable; the server loads the new model at next start.
Corrupt or malformed requests	Return 400 (counted as client errors); bad X-True-Label rejected before the model runs.
Fine-tuning forgets old data	Replay plus the clean-regression guard reject the candidate; the live model is kept and the batch is marked used so the next cycle sees new data.

7. Continual learning: opportunities and limits

- Implemented: log, monitor, trigger, fetch labelled rows, fine-tune, evaluate, conditional deploy, rollback, audit trail.
- Opportunities: active learning (label only low-confidence samples), scheduled retraining on real feedback, per-segment drift, shadow evaluation of candidates on live traffic, model versioning with A/B tests.
- Limits (honest): evaluation uses MNIST-derived drift, not real user drift; accuracy on live traffic is unobservable without labels, so low confidence is only a proxy; reload is single-process.
- Verification: 49 automated tests (API, logging, decisions, drift math, rollback), ruff lint, frontend tests and a Playwright end-to-end job run in GitHub Actions on every push.